Finance & LLM

Quand l’IA parle le langage de la règlementation bancaire

Soutenance de projet avec ActiveViam

Projet 32

Marie Leduc

Gabriel Anjos Moura

Trystan Aubertin

CentraleSupélec

Sommaire

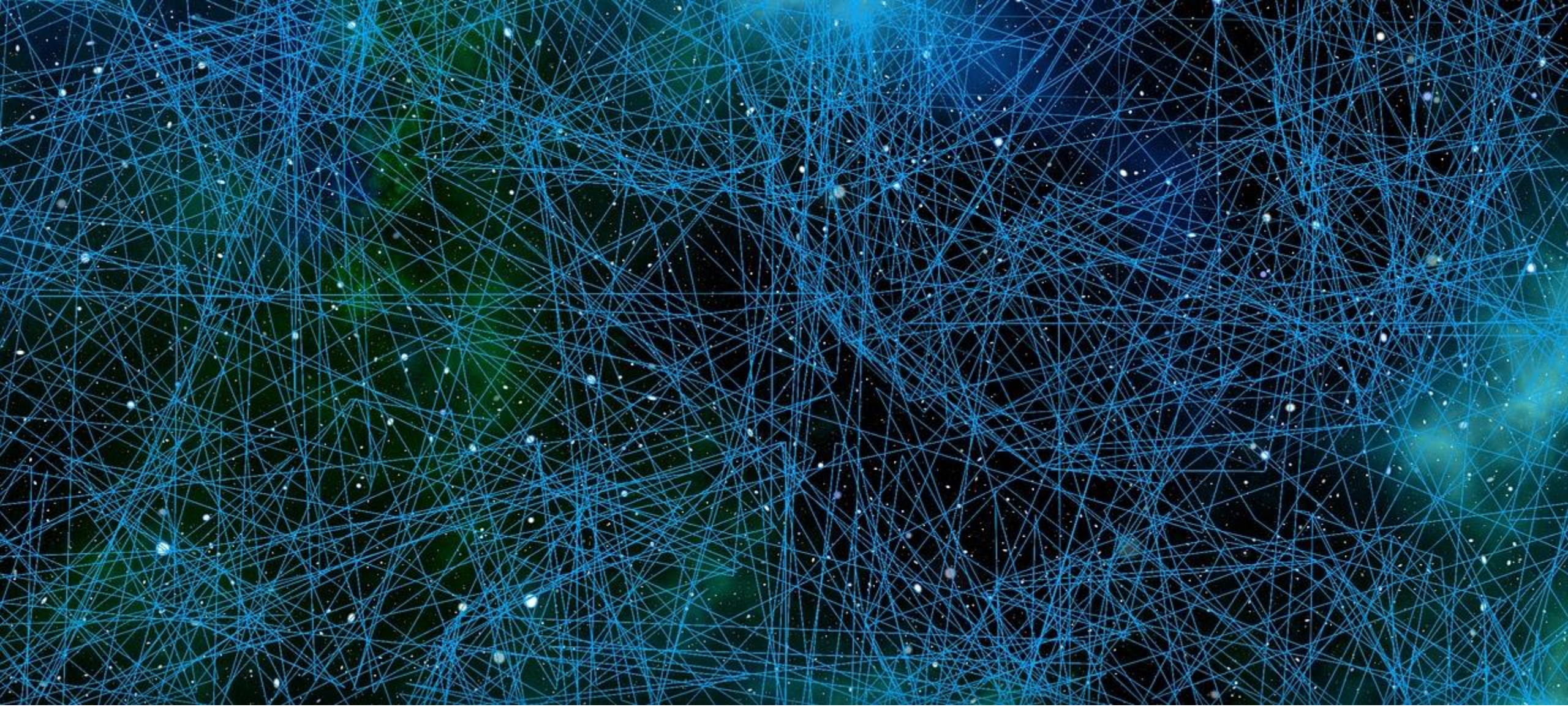
1 - Un contexte complexe

2 - Comment spécialiser un LLM ?

3 - RAG : la solution ?

4 - Evaluation et résultats de notre implémentation

5 - Perspectives



Introduction : contexte du projet

L'essor des LLMs

Un constat…

Existent depuis les années 1990s, mais trouvent leur *essor en 2017* à la sortie de "*Attention is all you need*" par des ingénieurs de Google.

Capables de *générer du texte*, de *comprendre du langage naturel*, de traiter
des informations, de produire des rapports…

Mais présentent un défaut majeur : les *hallucinations*

"résultats incorrects générés par des modèles d'IA"

Hallucinations de fidélité → réponse non fidèle au contexte
Hallucinations de faits → informations incorrectes
Fabrications de faits → invention de faits plausibles ou non

(Eleni Metheniti, Swarnadeep Bhar, Nicholas Asher. Une revue sur les hallucinations des LLMs)

Le domaine de la réglementation bancaire

Le domaine de la réglementation bancaire est un domaine spécifique qui présente des caractéristiques propres :

→ Textes longs et volumineux

→ Terminologie propre (abréviations, jargon financier)

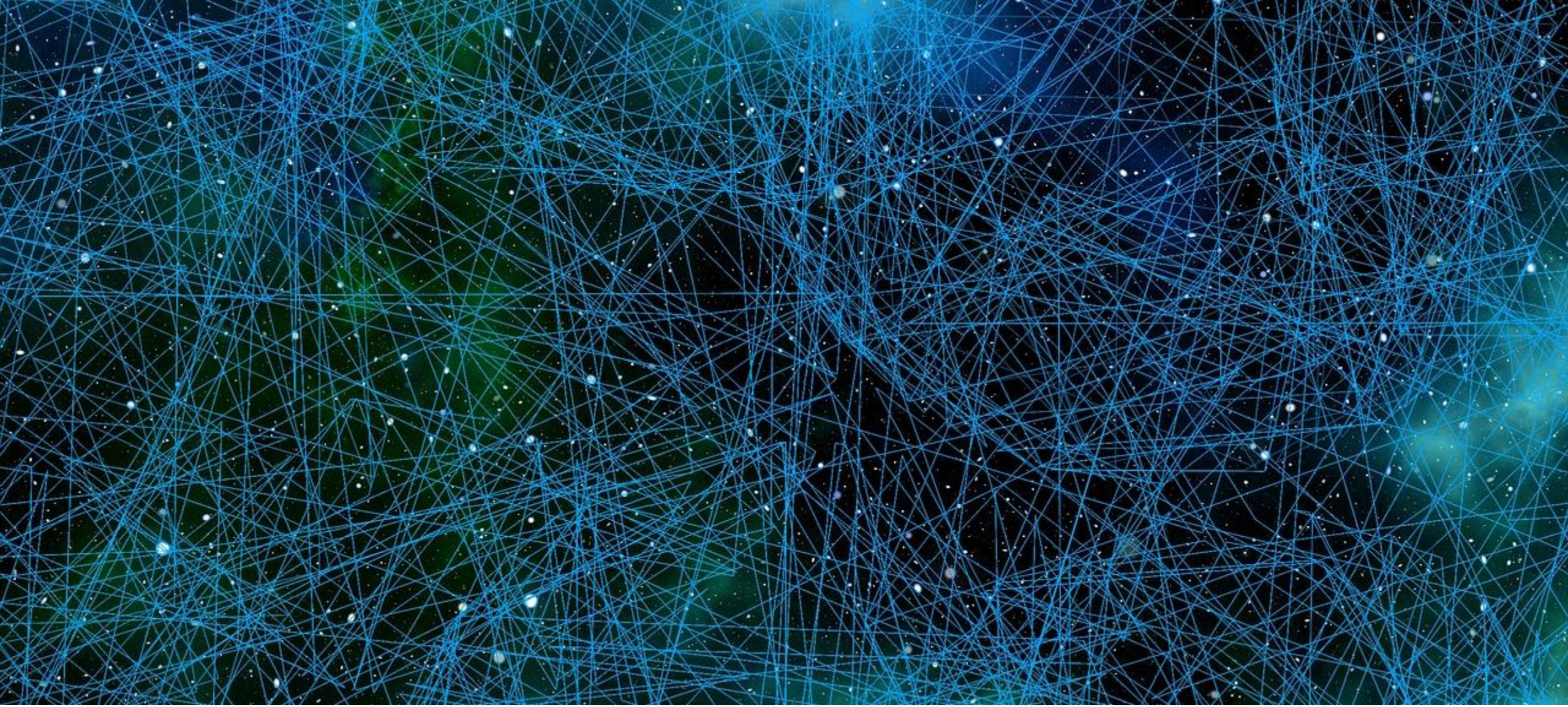
→ Exigences : traçabilité des rapports produits, véracité des informations, confidentialité des données

↓

Les exigences du domaine bancaire ne sont pas compatibles avec les défauts des LLMs généralistes

Problématique

Comment spécialiser un LLM afin qu'il puisse produire des réponses et des rapports techniques fiables dans un domaine réglementaire bancaire, tout en garantissant la traçabilité des informations et la maintenabilité du système?



Etat de l’art : comment spécialiser un LLM ?

Etat de l’art

La spécialisation des LLMs

Il existe différentes méthodes de spécialisation de LLMs :

Prompt Engineering
Mikhail Biriuchhinskii, Prompt Engineering : *L'art des requêtes efficaces*

Fine Tuning Complet
D.M. Anisuzzaman ; Jeffrey G. Malins ; Paul A. Friedman ; and Zachi I. Attia : *Fine-Tuning Large Language Models for Specialized Use Cases*

Retrieval Augmented Generation
Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra : *Retrieval Augmented Generation for Knowledge Intensive NLP Tasks*

Etat de l’art

Prompt Engineering, In-Context Learning

Principe : orientation du comportement du modèle via des instructions et des exemples inclus dans le prompt.

Avantages de la méthode : simplicité, aucun entraînement

Désavantages de la méthode :

- → Connaissance limitée à celle du modèle
- → Couverture du domaine non garantie
- → Illustration contrainte par la fenêtre du contexte
- → Traçabilité peu fiable

Etat de l’art

Fine-tuning complet

Principe : poursuite de l’entraînement d’un modèle sur un corpus spécifique au domaine, afin d’adapter son comportement.

Avantages de la méthode : utile pour apprendre du vocabulaire de domaine

Désavantages de la méthode :

- → Coûteuse en calculs
- → Mises à jour difficiles (nécessite un nouveau fine-tuning à chaque fois)
- → Sujette à des oublis
- → Justifications des réponses non automatiques

Etat de l’art

Retrieval-Augmented Generation (RAG)

Principe : **récupération de documents (Retrieval)** pertinents à partir d’une requête utilisateur, puis génération d’une **réponse à partir de ces documents** (Augmented Generation).

Avantages de la méthode :

- Fiabilité des résultats
- Réponses limitées au corpus fourni
- Pas de nécessité de ré-entraînement à l'ajout de documents

Désavantages de la méthode :

- Temps de réponse augmenté
- Fenêtres contextuelles limitées

Etat de l’art

Retrieval-Augmented Generation (RAG) - Fonctionnement

Retrieval

1. Encodage de la requête sous forme de vecteur (appelé ***embedding***)
2. Recherche les ***k parties de documents les plus similaires*** à la requête dans une base vectorielle
3. Retour de ***fragments textuels*** accompagnés de leurs métadonnées pour le contexte

Augmented Generation

1. Injection des documents récupérés dans le ***prompt d'un LLM***
2. Génération d'une réponse conditionnée par le ***contexte fourni***

Etat de l’art

Retrieval-Augmented Generation (RAG) - Pipeline complet

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

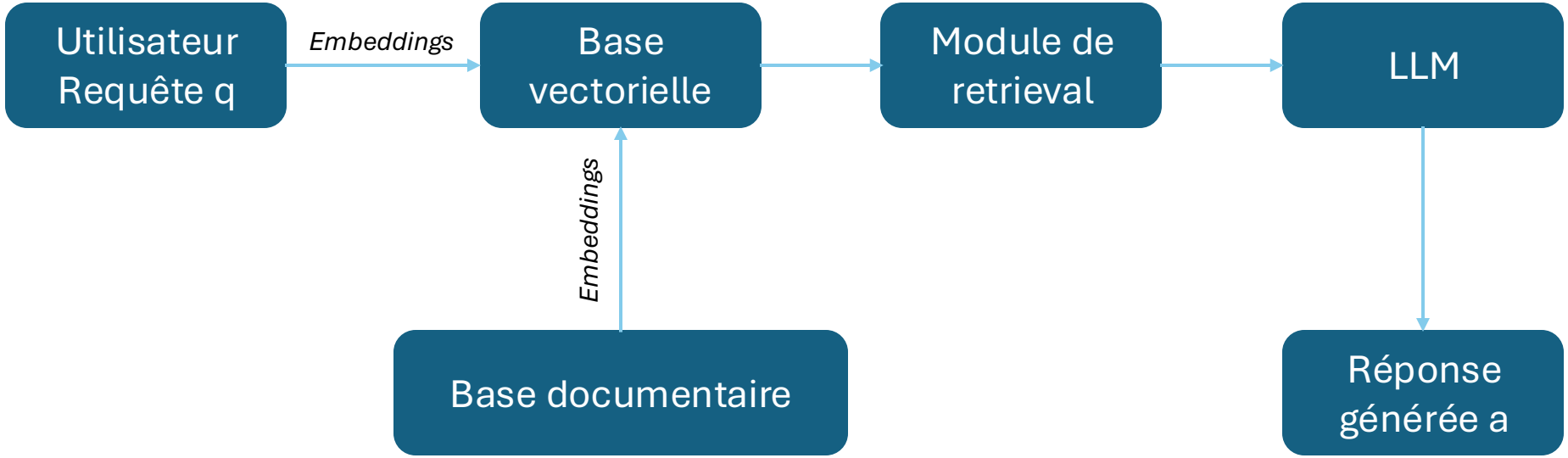
Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Etat de l’art

Retrieval-Augmented Generation (RAG) - RAG naïf

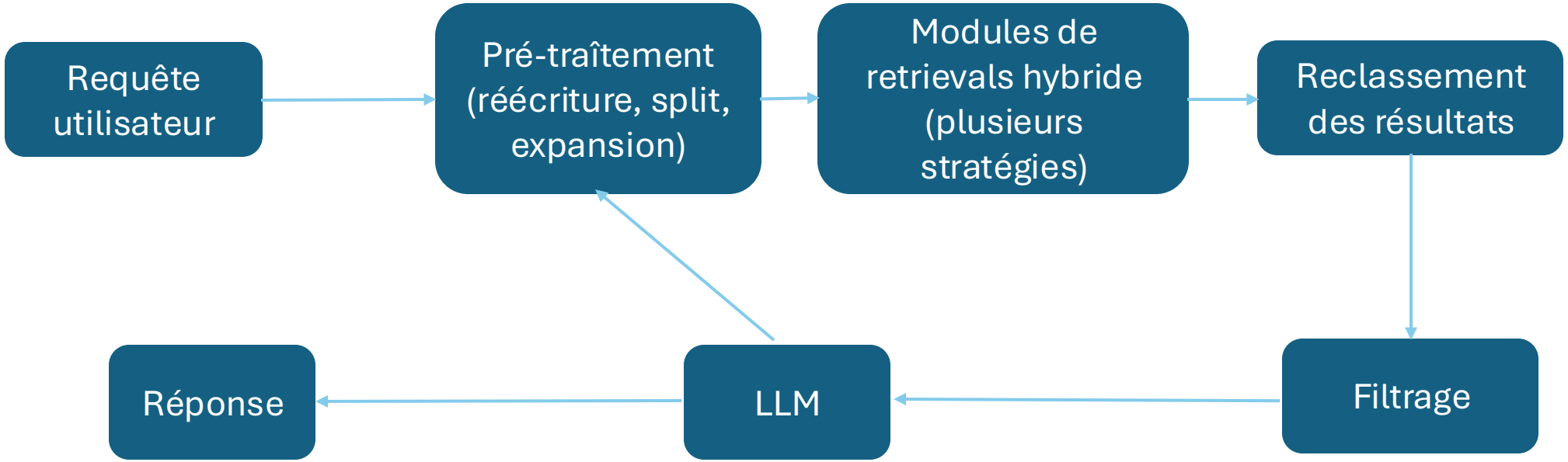
RAG naïf



Etat de l’art

Retrieval-Augmented Generation (RAG)

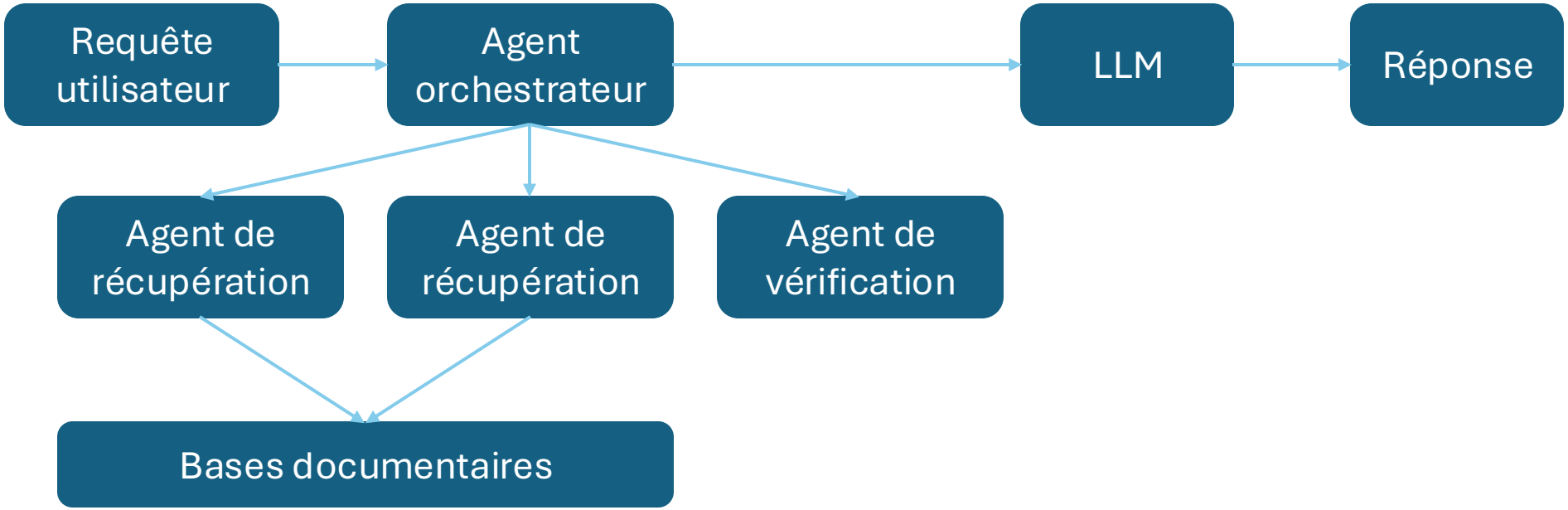
<u>RAG Modulaire</u>

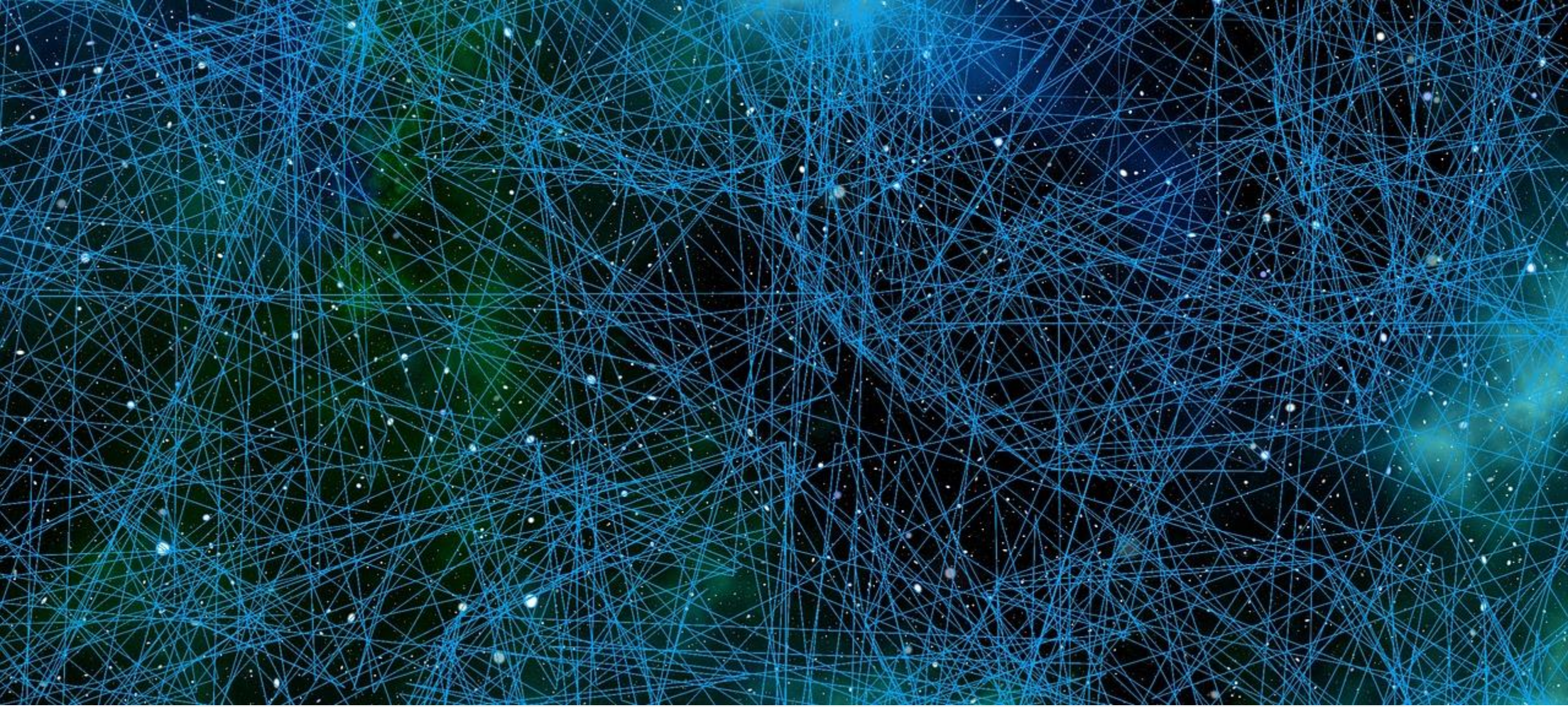


Etat de l’art

Retrieval-Augmented Generation (RAG)

RAG Agentique





RAG : la solution ?

Architecture du projet

Rappel : RAG - Pipeline complet

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

Indexation et structuration des données

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Extraction du texte page par page

Chunking

Enrichissement via des métadonnées (nom de l'entreprise, année du rapport, fichier source)

Embedding

Insertion dans la base vectorielle

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

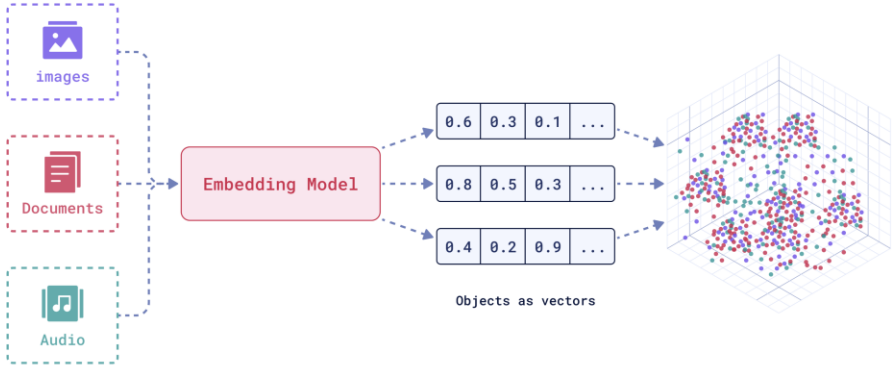
Embeddings

Etape de transformation des segments textuels des rapports PDF en vecteurs numériques (qu'on appelle embeddings)

→ Permet la recherche sémantique dans la base ChromaDB : comparaison entre vecteurs dans un espace commun

→ Implémentation de 3 embeddings :

- ***TF IDF + SVD***
- ***Word2Vec***
- ***SentenceTransformers***



→ Permet la comparaison entre différentes méthodes : statistique, entrainée sur le corpus, et une approche neuronale pré-entraînée.

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

Embeddings : TF IDF + SVD

Train Frequency – Inverse Document Frequency + Singular Value Decomposition
Entraînement local

Représentation du document par un vecteur TF - IDF

Mesure de l'importance d'un terme dans un document relativement à tout le corpus

Projection de la matrice TF – IDF dans un espace de dimension réduite via SVD

Compression de l'information utile

Réduction du coût de stockage

Capture des corrélations latentes entre termes

Avantages :

- Simplicité
- Rapidité d'entraînement
- Exécution légère
- Intepretable (dérive d'une représentation lexicale classique)

Inconvénients :

- Dépend du vocabulaire entre requête et documents -> gère mal les synonymes

CentraleSupélec

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

Embeddings : Word2Vec

Word2Vec
Entraînement local

Tokenisation des pages en mots simples

Entraînement d'un modèle skip-gram

Encodage des mots prenant en compte le voisinage

Agrégation pour encoder un segment textuel complet

Chaque mot correspond à la représentation finale proportionnellement à son importance dans le corpus

Avantages :

- Capture des similitudes entre mots qui ne coexistent pas à l'identique dans les documents (synonymes)

Inconvénients :

- Performances dépendent de la qualité du corpus disponible : ici le corpus contient des abréviations et des mots rares, qui sont moins bien modélisés…

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

Embeddings : SentenceTransformers

SentenceTransformers
Entraînement externe sur un modèle déjà appris : on exploite les vecteurs pour les tâches de similarité sémantique

Texte fourni → Génération d'un vecteur dense de dimension fixe (pas d'entraînement local)

Avantages :

- Capture de similarité sémantique plus profonde que les autres modèles : pas de limitation à la concurrence de mots (rapprochement de formulations différentes ayant le même sens)

Inconvénients :

- Lourd en mémoire et en calcul
- Dépendance à un corpus externe moins spécifique à notre domaine

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

Retrieval

Sélection des fragments les plus pertinents pour répondre à la requête donnée.

Encodage de la requête utilisateur en vecteur

↓

Recherche des k vecteurs les plus proches dans la base

↓

Retour des fragments correspondants avec leur métadonnées

Le retrieval est hybride : on combine les résultats issus de différents embeddings (dans notre cas SentenceTransformers et TF IDF)

La combinaison des résultats se fait via une Reciprocal Rank Fusion : combinaison de différents classements issus de différents modèles d'embeddings

$$RRF(d) = \sum_{i=1}^{n} \frac{1}{k+rank_i(d)}$$

Avec le document, rank le rang du document, et k un paramètre de lissage

Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009) : *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods.*

Pré-traitement des documents (création d'embeddings sur les documents)

Embedding de la requête utilisateur

Récupération des embeddings issus de documents les plus proches

Injection des embeddings récupérés dans le LLM

Génération d'une réponse

Architecture du projet

Agent RAG

Combinaison des résultats du retrieval avec un LLM pour produire une réponse finale

mécanisme d'orchestration	Appel d'une fonction de recherche qui interroge la base de données et retourne les fragments pertinents
Fonctionnement général	1. Analyse de la question utilisateur 2. Appel de la fonction de recherche avec une requête reformulée 3. Réception de contextes correspondants 4. Génération d'une réponse basée sur ces contextes.
Contraintes imposées	Obligation d'utiliser l'outil de recherche Limitation du nombre d'itérations Contrainte sur le format de sortie

Architecture du projet

Métriques d'évaluation

Hit@K

Mesure la capacité du retrieval à récupérer un contexte contenant la bonne réponse

Accuracy

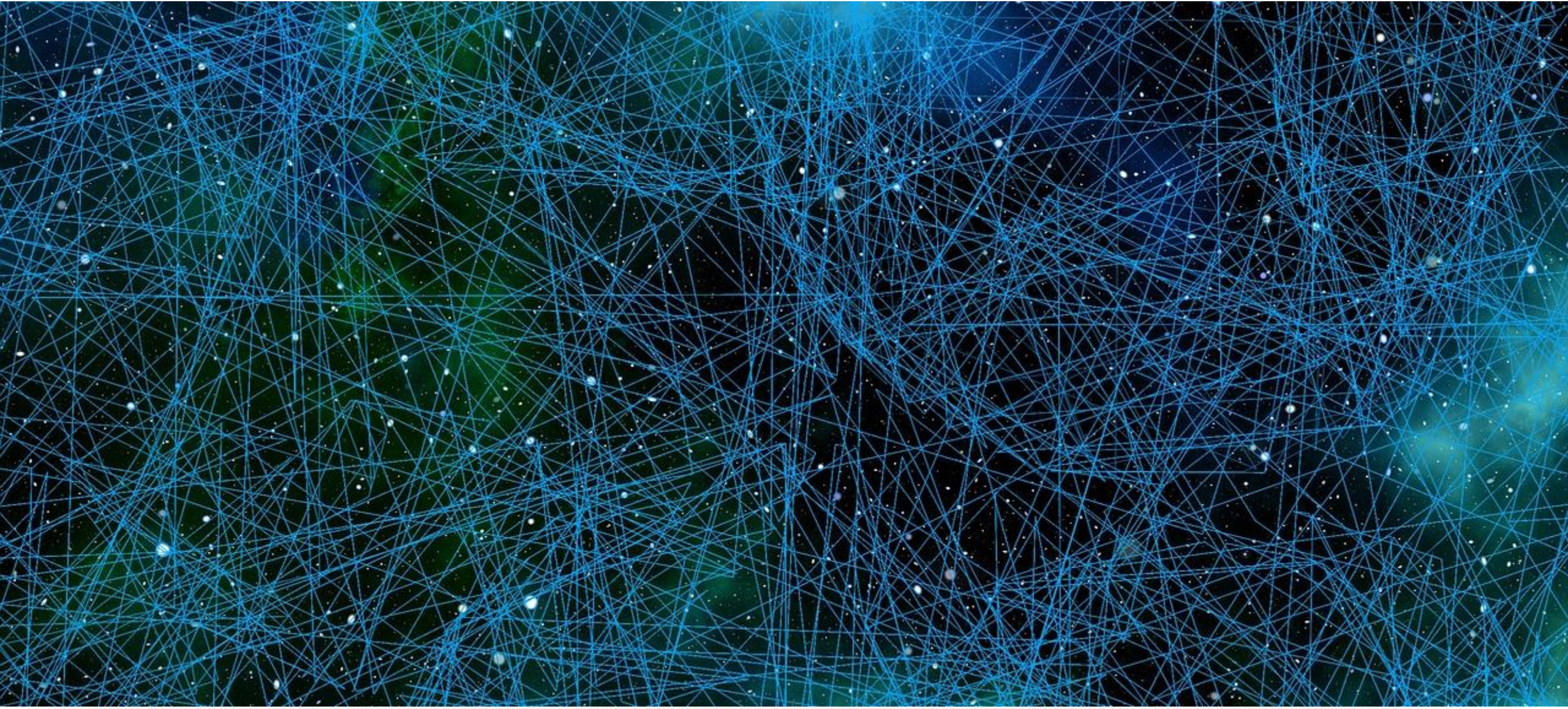
Mesure la proportion de réponses correctes générées

Matching robuste

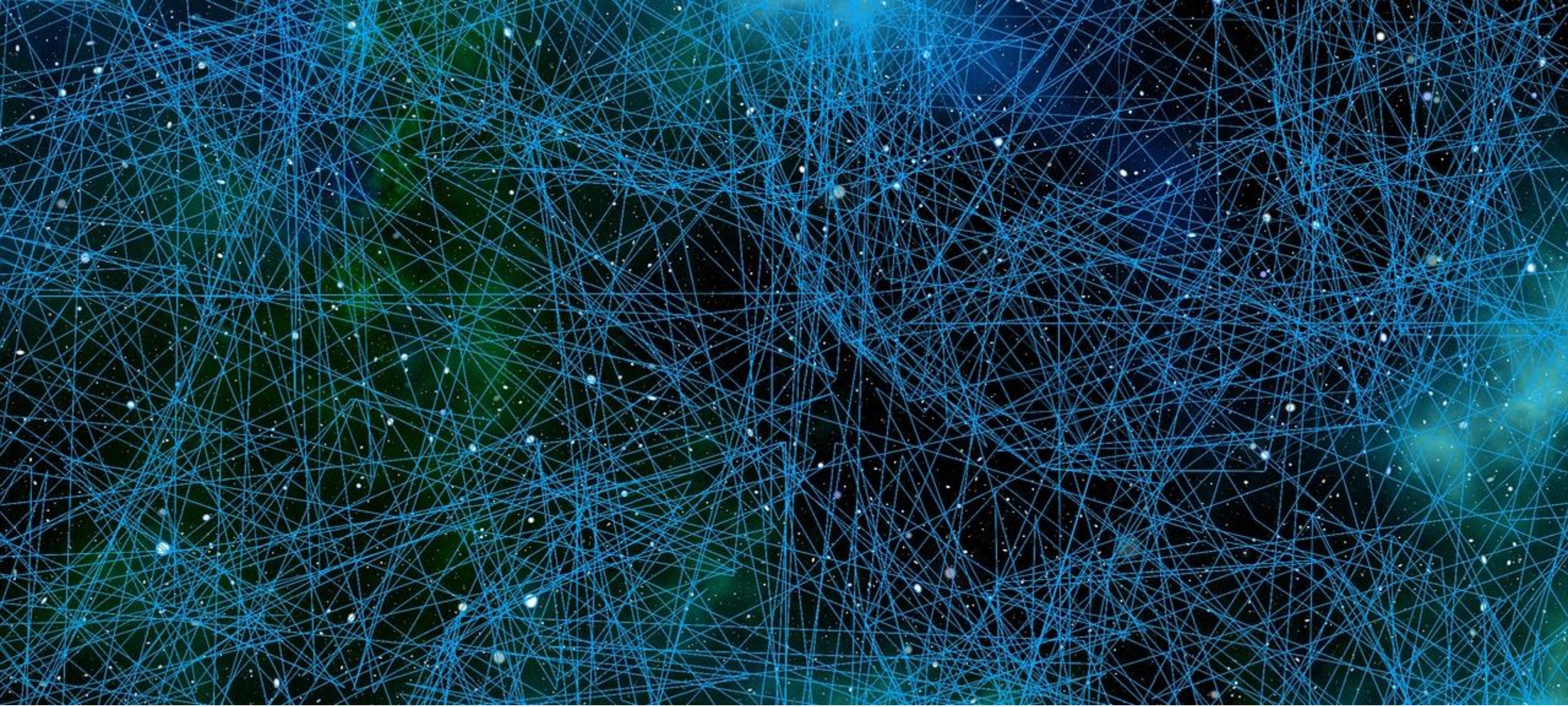
Evalue la correspondance numérique en tenant compte des formats et des unités

Analyse qualitative

Inspection d'erreurs pour identifier les limites du système

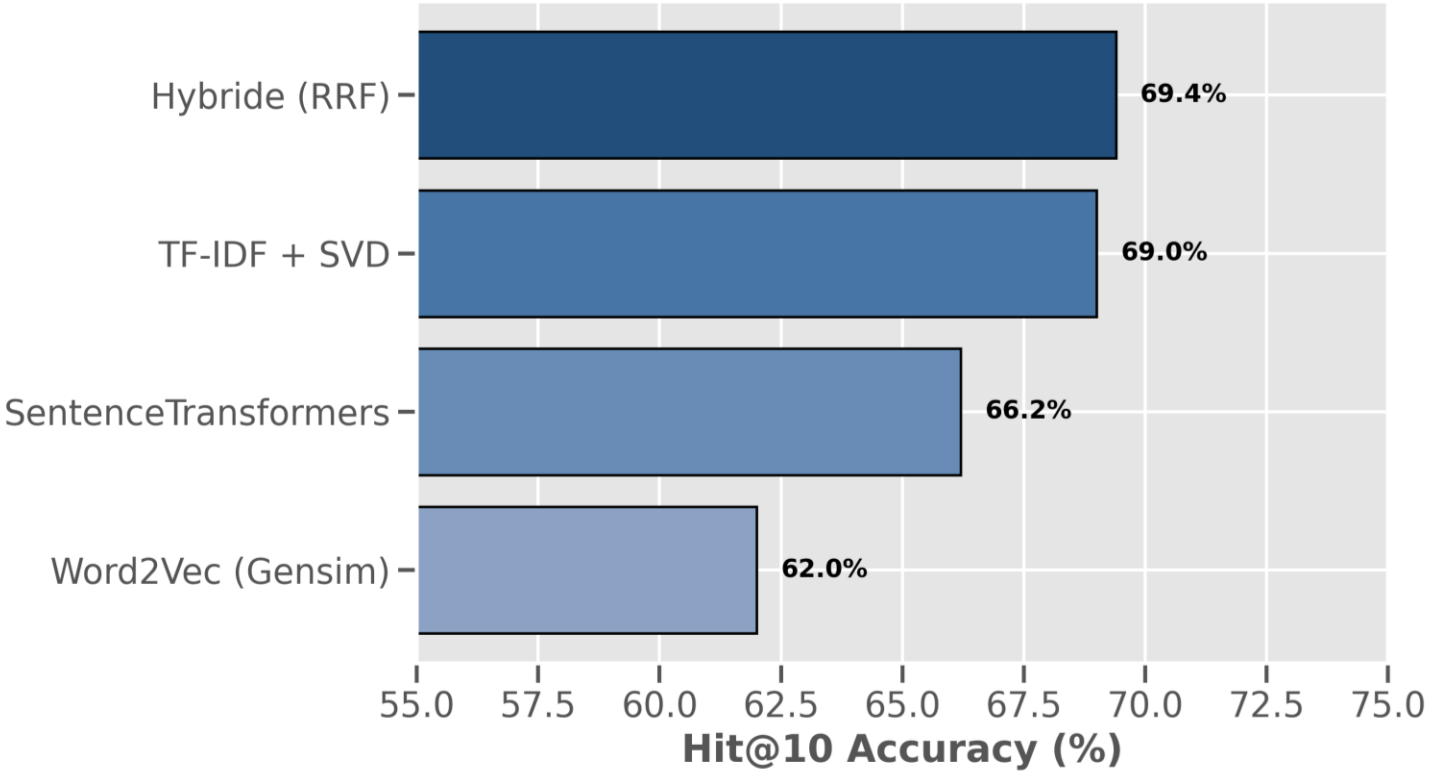


Interface



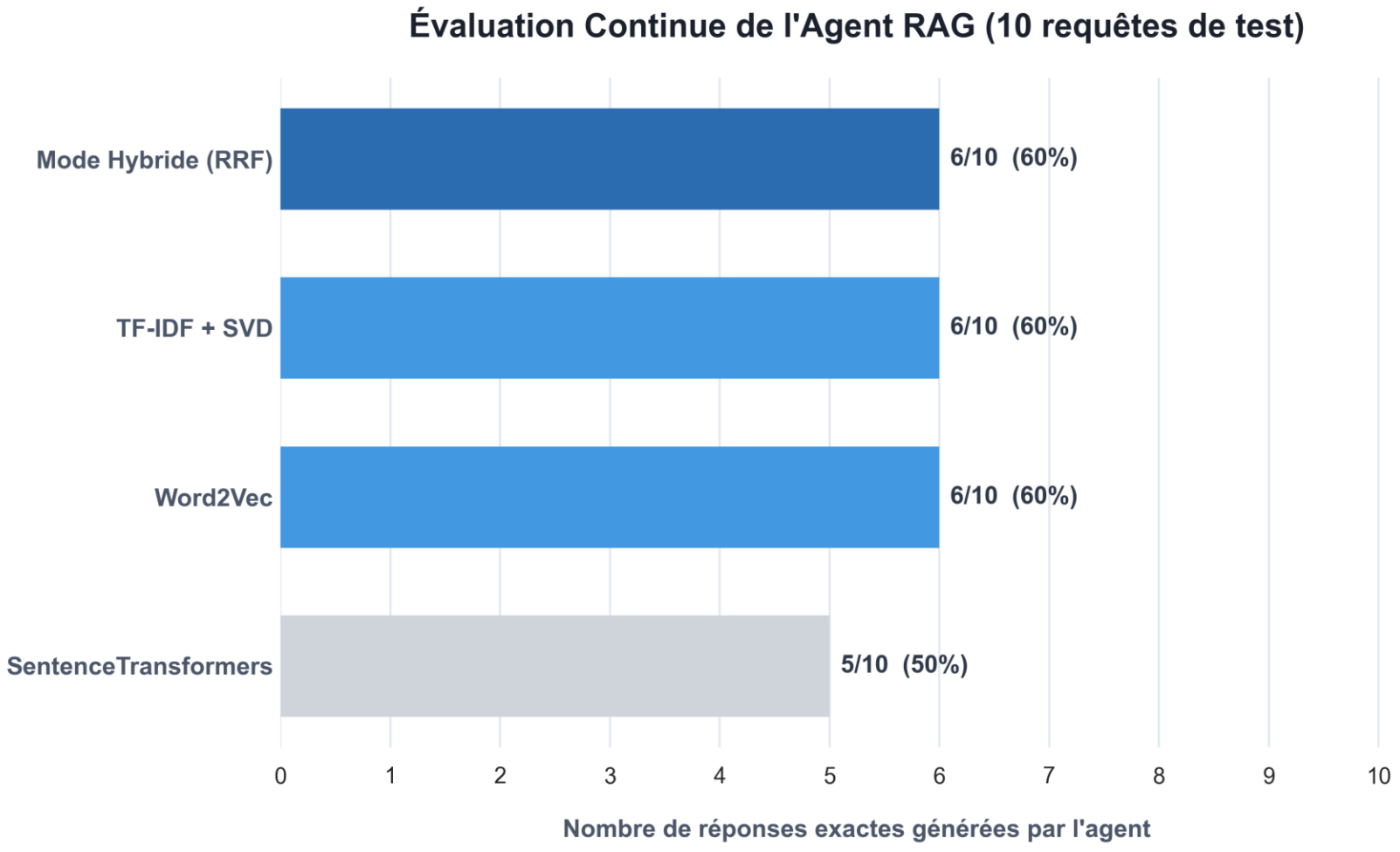
Résultats

Comparaison des Algorithmes de Retrieval



- Le mode Hybrid surpasse tous les autres (Hit@10 ≈ 69%).
- Le TF-IDF+SVD est performant sur les termes techniques financiers

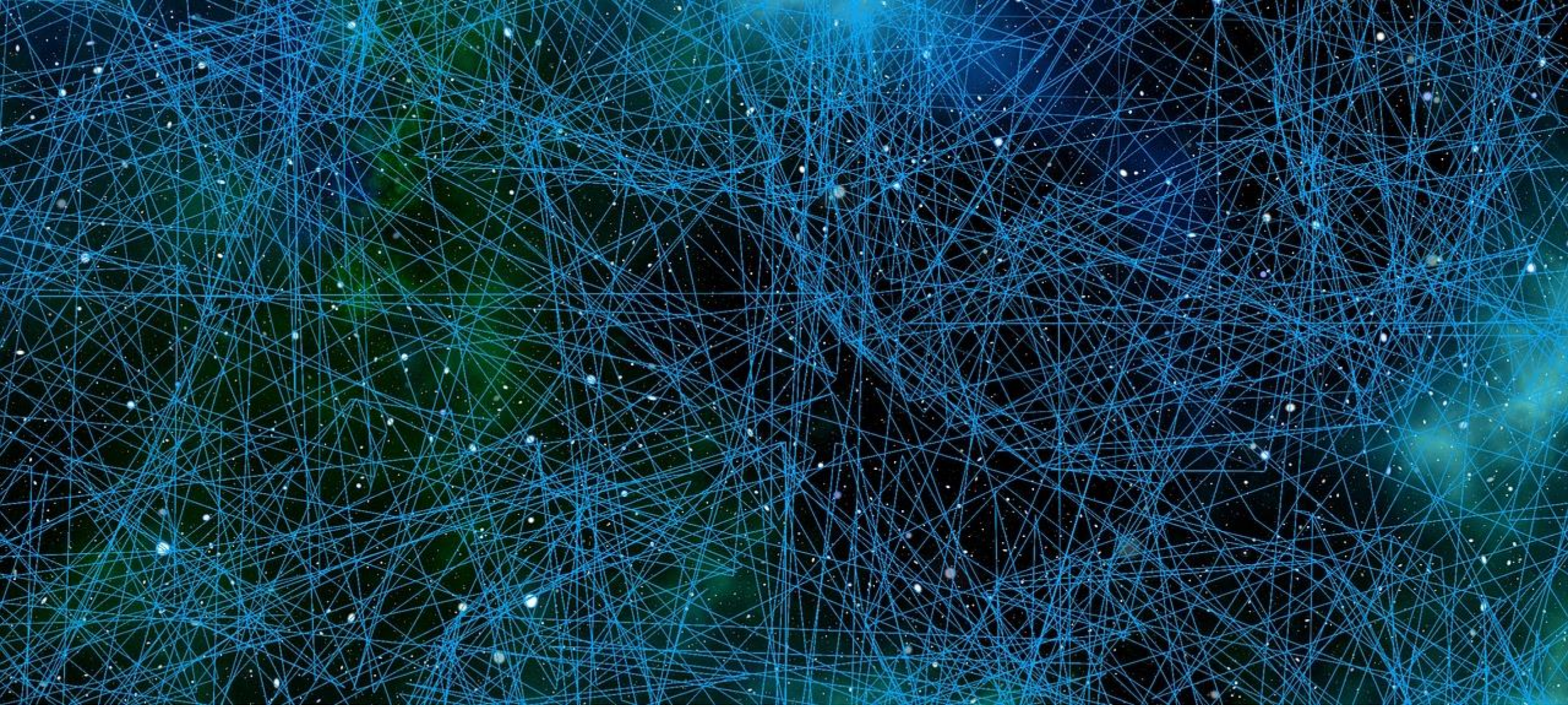
Précision Générative de l'Agent RAG



Note : Performances de l'agent complet après retrieval + génération du LLM.

- **Scores lissés (60%) :**
L'échantillon de test réduit par les quotas d'API masque la supériorité attendue du mode hybride.

- **Généralisation vs Overfitting :**
Les embeddings peinent sur nos tableaux hyper-spécifiques, mais forcer une méthode d'extraction sur-mesure causerait un surapprentissage. Le RAG garantit la flexibilité.



Perspectives

Pour aller plus loin…

Tester de nouveaux embeddings :

Ici 3 embeddings testés, mais on pourrait continuer d'approfondir le modèle en testant de nouveaux embeddings (plus simples ou plus complexes) et étudier leurs effets sur le modèle

Tester de nouveaux modes de retrieval :

Nous avons testé un retrieval hybride, mais nous avions testé le Page Index, PDF Raw, plus simples au début de notre projet (qui performaient mal), il est envisageable d'en tester d'autres afin d'améliorer la robustesse de notre RAG

Etendre le dataset :

Les entraînements ont été réalisés sur le dataset DataRetriever (Kaggle) mais il est envisageable d'étendre ce dataset à des datasets plus conséquents (FinQA par exemple)

Merci !

Soutenance de mi-parcours

Projet 32

Marie Leduc

Gabriel Anjos Moura

Trystan Aubertin

Références

- RAGAS. (2023).
  RAGAS: Automated Evaluation of Retrieval-Augmented Generation.
- Asai, A., et al. (2023).
  Self-RAG: Learning to Retrieve, Generate, and Critique.
- Basel Committee on Banking Supervision.
  Basel III: International Regulatory Framework.
- Dettmers, T., et al. (2023).
  QLoRA: Efficient Finetuning of Quantized Large Language Models.
- Gao, L., et al. (2022).
  HyDE: Hypothetical Document Embeddings.
- Lewis, P., Perez, E., Piktus, A., et al. (2020).
  Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS
- Metheniti, E., Bhar, S., & Asher, N. (2025).
  Une revue sur les hallucinations des LLM. TALN.
- Robertson, S., & Zaragoza, H. (2009).
  The Probabilistic Relevance Framework: BM25 and Beyond.